Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Update Sets

Abstract

This project focuses on enhancing data integrity and consistency in ServiceNow by enforcing mandatory fields using UI Policies and managing configuration changes through Update Sets. UI Policies allow dynamic control over form behavior without scripting, ensuring that users provide a essential information before submitting records. Update Sets are used to track and migrate configuration changes from one instance to another, ensuring a structured and error-free deployment process. The project demonstrates how these features improve workflow efficiency, reduce data errors, and support smooth application lifecycle management.

Objective

- To understand and implement UI Policies in ServiceNow.
- To enforce mandatory fields dynamically based on conditions.
- To ensure data accuracy and completeness in forms.
- To learn the concept of Update Sets for change management.
- To migrate configurations between instances safely.
- To minimize errors during deployment and maintain system consistency.

Technology Description

1. ServiceNow Platform

A cloud-based IT Service Management (ITSM) platform used for automating business processes and managing workflows.

2. UI Policies

- Used to control form behavior dynamically.
- Can make fields mandatory, visible, or read-only.
- Works without scripting (low-code approach).
- Example: Making “Resolution Notes” mandatory when a ticket is closed.

3. UI Policy Actions

- Define what happens to fields (mandatory, visible, read-only).
- Linked with UI Policies to enforce rules.

4. Update Sets

- A feature used to capture configuration changes.
- Helps in moving customizations from Development → Test → Production.
- Ensures version control and safe deployment.

5. Instances in ServiceNow

- Development Instance – Where changes are created.
- Test Instance – Where changes are validated.
- Production Instance – Live environment.

Detailed Execution of Project Phases

Phase 1: Requirement Analysis

- Identify the need for mandatory fields in forms.
- Example: Incident form requires “Work Notes” before closing a ticket.
- Define conditions under which fields should become mandatory.

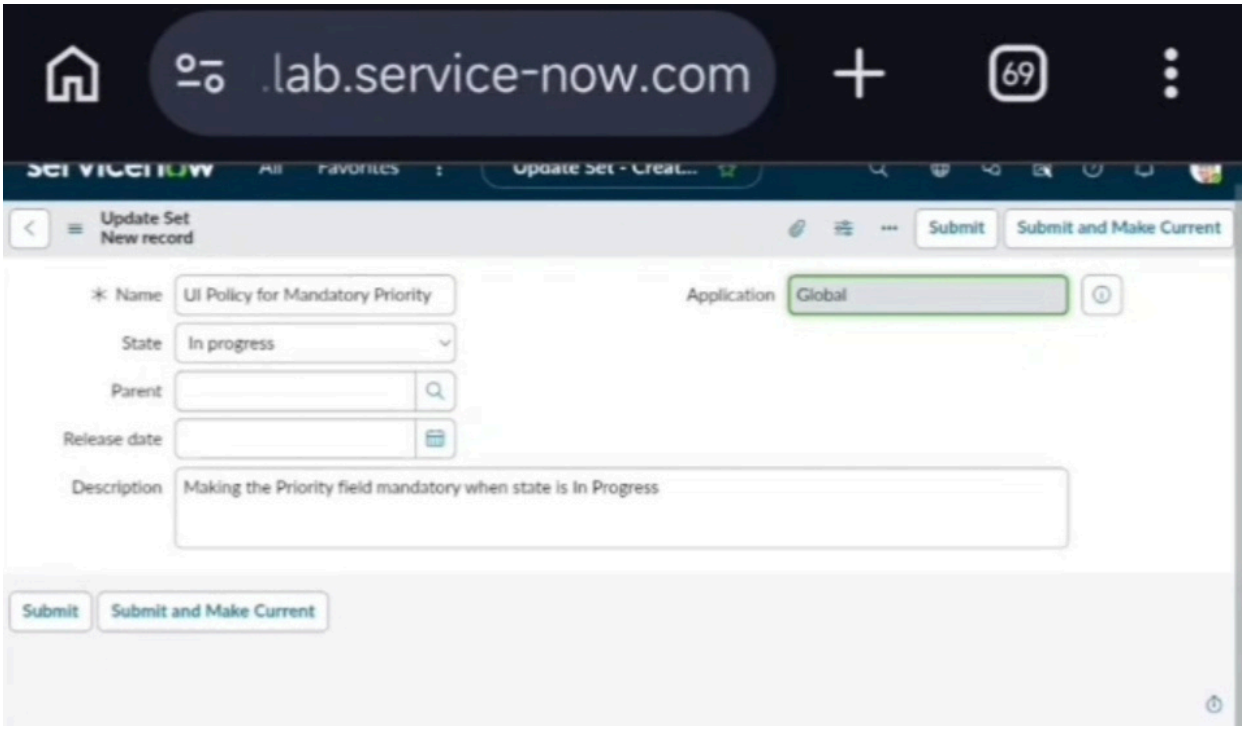
Phase 2: Design

- Plan UI Policy rules:
- Condition: State = Closed
- Action: Make “Resolution Notes” mandatory
- Design workflow for change migration using Update Sets.

Phase 3: Implementation

Step 1: **Create Update Set - Update Sets**

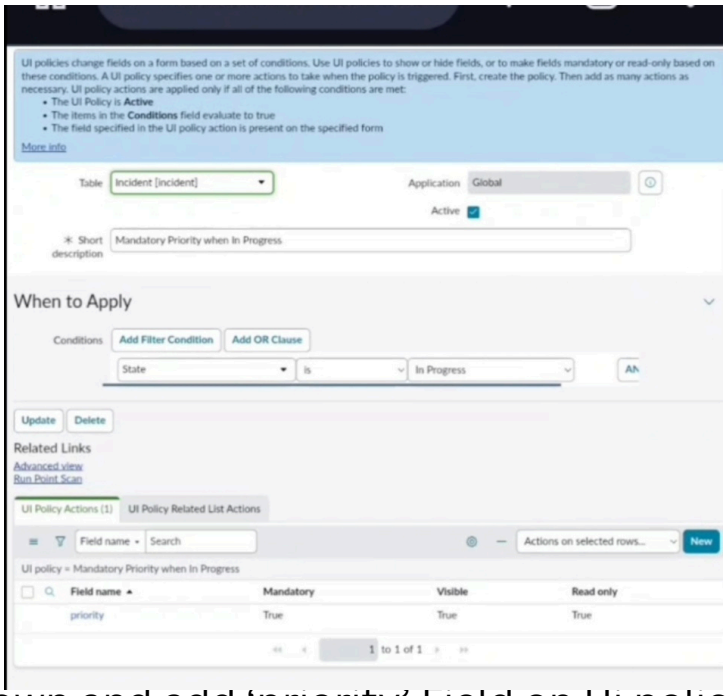
- Open service now.
- Click on All >> search for Update Sets
- Select Local Update Sets
- Click on new.
- Fill the details below and make current.



- Click submit.

Step 2: **Create UI Policy - Ui Policy**

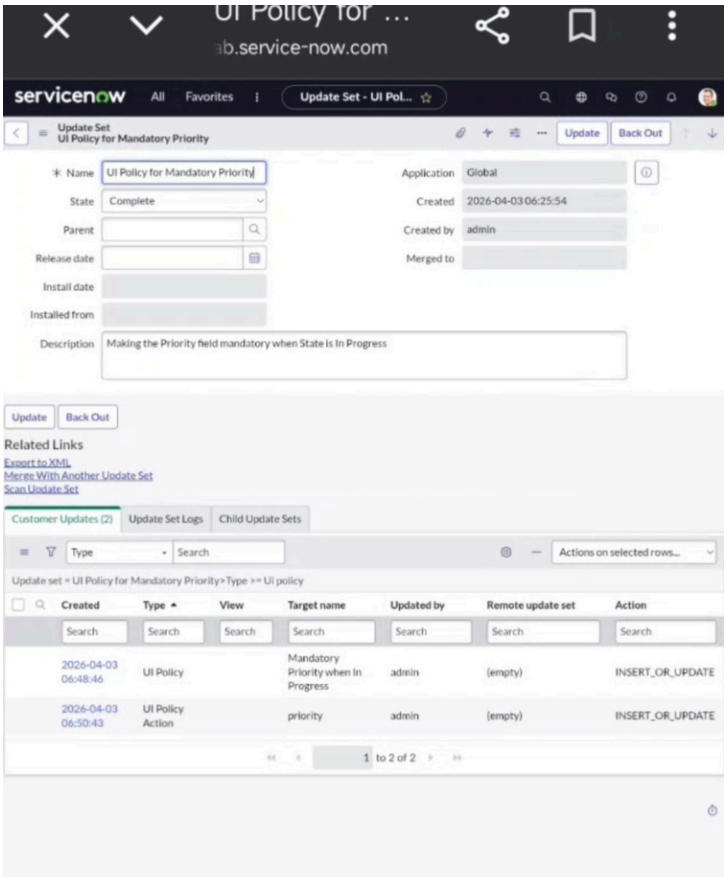
- Open service now.
- Click on All >> search for ui policies
- Navigate to System UI >> UI Policies
- Click on new
- Fill the following details and Save



- Scroll Down and add ‘priority’ Field on UI policy Actions
- Click on submit.

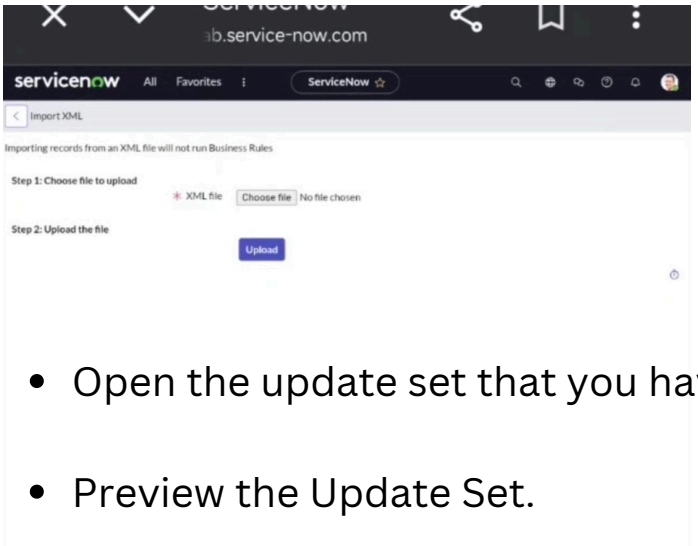
Step 3: **Check Update Set**

- Navigate to the System Update Sets > Local Update Sets.
- Open the ‘Ui policy for mandatory priority’ Update Set.
- Verify that your UI Policy and UI Policy Action are listed.

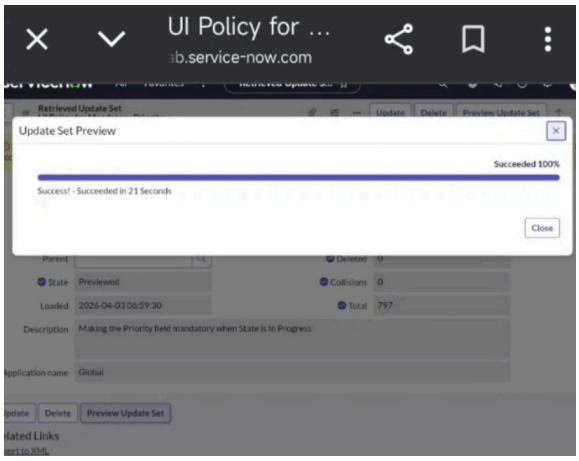


Step 4: **Export and Import the Update Set**

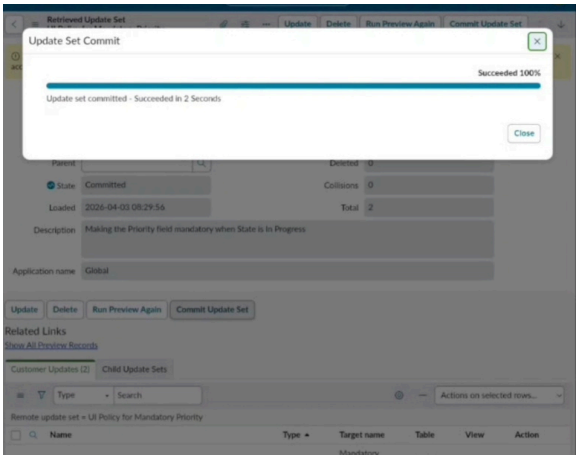
- Set State to Complete and Click Export to XML on the Update Set record.
- Download the XML file.
- Now Open Another Instance to Export Update Set.
- Go to System Update Sets >> Retrieved Update Sets.
- Click on the Import update set from XML and Select File and Upload



- Open the update set that you have created in your instance here.
- Preview the Update Set.

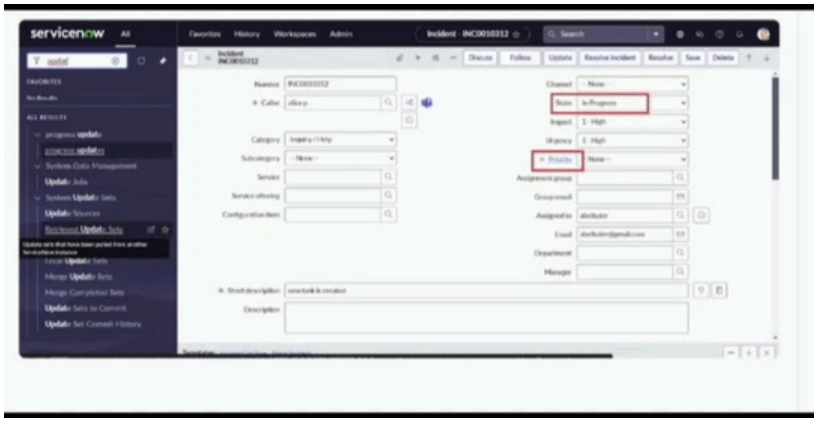


- Commit the Update Set.



Phase 4: **Testing**

- Verify:
- Mandatory fields are enforced correctly.
- UI behaves as expected.
- Update set migration works without conflicts.



Phase 5: **Deployment**

- Move update set to production.
- Validate functionality in live environment.

Phase 6: **Maintenance**

- Monitor performance.
- Modify UI Policies if requirements change.

Conclusion

This project demonstrates how UI Policies can effectively enforce mandatory fields and improve data quality without complex scripting. Update Sets ensure safe and structured migration of configurations across instances. Together, these features enhance reliability, reduce manual errors, and streamline ServiceNow development and deployment processes.

Future Scope

- Implement Client Scripts for advanced validations.
- Integrate with Business Rules for backend validation.
- Automate deployments using CI/CD pipelines.
- Use Flow Designer for advanced workflow automation.
- Apply similar concepts to other modules like Change, Problem, and Request Management.
- Enhance user experience with dynamic forms and AI-based suggestions.